

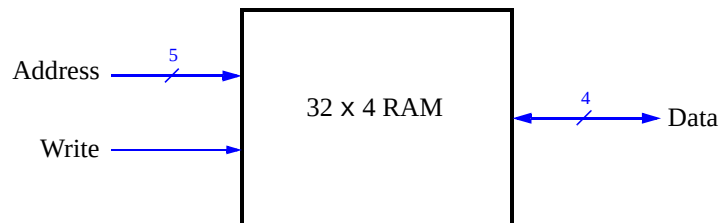
Digital Systems LAB

Exercise 7: Memory Blocks

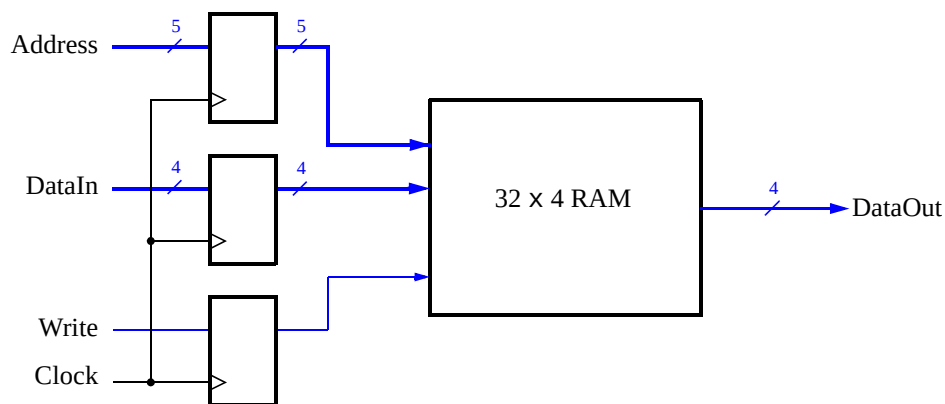
In computer systems it is necessary to provide a substantial amount of memory. If a system is implemented using FPGA technology it is possible to provide some amount of memory by using the memory resources that exist in the FPGA device. In this exercise we will examine the general issues involved in implementing such memory.

A diagram of the random access memory (RAM) module that we will implement is shown in Figure 1a. It contains 32 four-bit words (rows), which are accessed using a five-bit *address* port, a four-bit *data* port, and a *write* control input.

The FPGAs that are included on the Intel FPGA DE10-Lite, DE0-CV, DE1-SoC, and DE2-115 boards provide dedicated memory resources. The MAX 10 FPGA on the DE10-Lite, and Cyclone IV FPGA on the DE2-115 contain dedicated memory resources called *M9K blocks*. The Cyclone V FPGA on the DE0-CV and DE1-SoC boards have *M10K blocks*. Each M9K block contains 9216 memory bits, while each M10K block contains 10240 memory bits. Both M9K and M10K blocks can be configured to implement memories of various sizes. A common term used to specify the size of a memory is its *aspect ratio*, which gives the *depth* in words and the *width* in bits (depth x width). In this exercise we will use an aspect ratio that is four bits wide, and we will use only the first 32 words in the memory. Although the M9K and M10K blocks support many other modes of operation, we will not discuss them here.



(a) RAM organization



(b) RAM implementation

Figure 1: A 32 x 4 RAM module.

There are two important features of the M9K and M10K blocks that have to be mentioned. First, they includes

registers that can be used to synchronize all of the input and output signals to a clock input. The registers on the input ports must always be used, and the registers on the output ports are optional. Second, the blocks have separate ports for data being written to the memory and data being read from the memory. Given these requirements, we will implement the modified 32 x 4 RAM module shown in Figure 1b. It includes registers for the *address*, *data input*, and *write* ports, and uses a separate unregistered *data output* port.

Part I

Commonly used logic structures, such as adders, registers, counters and memories, can be implemented in an FPGA chip by using prebuilt modules that are provided in *libraries*. In this exercise we will use such a module to implement the memory shown in Figure 1b.

1. Create a new Quartus project to implement the memory module.
2. To open the IP Catalog in the Quartus software click on Tools > IP Catalog. In the IP Catalog window choose the *RAM: 1-PORT* module, which is found under the Basic Functions > On Chip Memory category. Select VHDL as the type of output file to create, give the file the name *ram32x4.vhd*, and click OK. As shown in Figure 2 specify a memory size of 32 four-bit words. Select M9K if your DE-series board has a MAX 10 or Cyclone IV FPGA, otherwise select M10K. Also on this screen accept the default setting to use a single clock for the memory's registers, and then advance to the page shown in Figure 3. On this page *deselect* the setting called 'q' output port under the category Which ports should be registered?. This setting creates a RAM module that matches the structure in Figure 1b, with registered input ports and unregistered output ports. Accept defaults for the rest of the settings in the Wizard, and click the Finish button to exit from this tool. Examine the *ram32x4.vhd* VHDL file which defines the following subcircuit:

```
ENTITY ram32x4 IS
    PORT ( address : IN    STD_LOGIC_VECTOR (4 DOWNT0 0);
          clock    : IN    STD_LOGIC := '1';
          data      : IN    STD_LOGIC_VECTOR (3 DOWNT0 0);
          wren      : IN    STD_LOGIC ;
          q         : OUT   STD_LOGIC_VECTOR (3 DOWNT0 0) );
END ram32x4;
```

3. Instantiate this subcircuit in a top-level VHDL file that includes appropriate input and output signals for the memory ports given in Figure 1b.
4. Compile the circuit. Observe in the Compilation Report that the Quartus Compiler uses 128 bits in one of the FPGA memory blocks to implement the RAM circuit.
5. Simulate the behavior of your circuit and ensure that you can read and write data in the memory. An example simulation output is given in Figure 4.

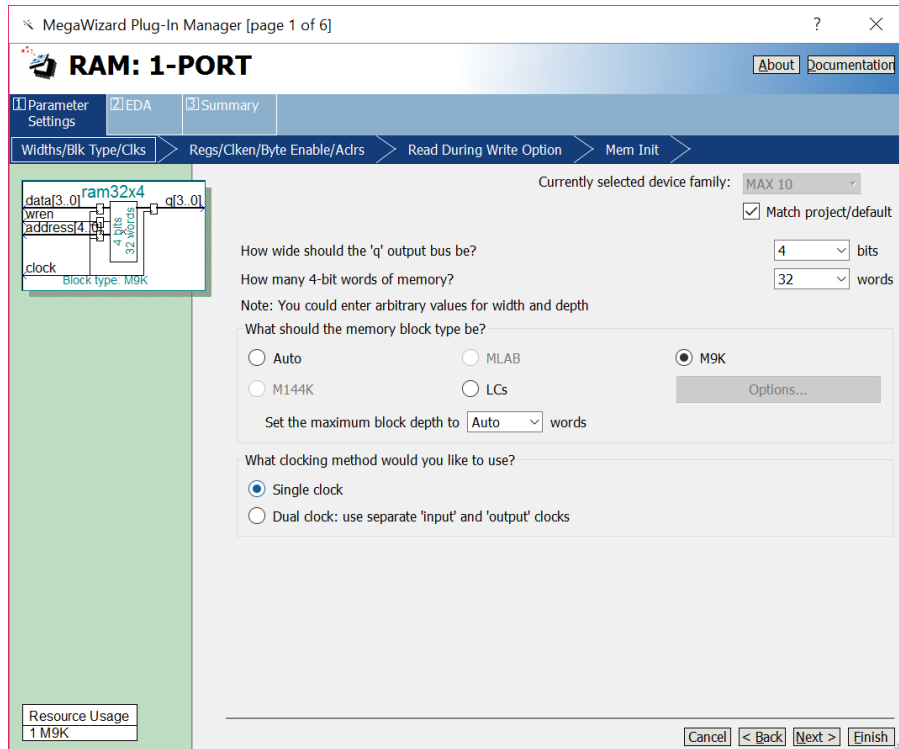


Figure 2: Configuring the size of the memory module.

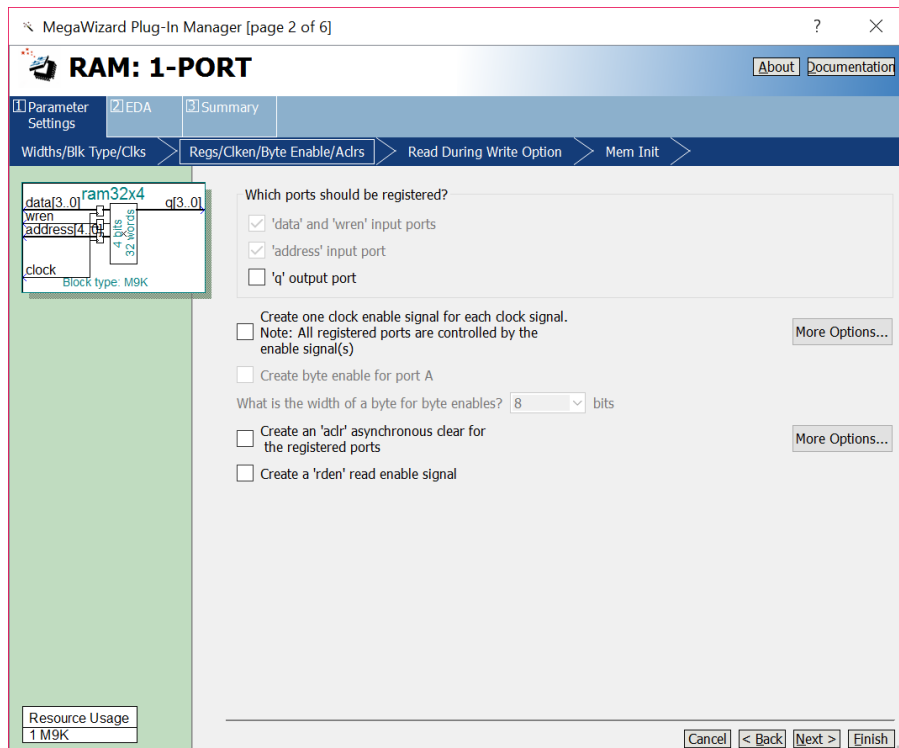


Figure 3: Configuring input and output ports.

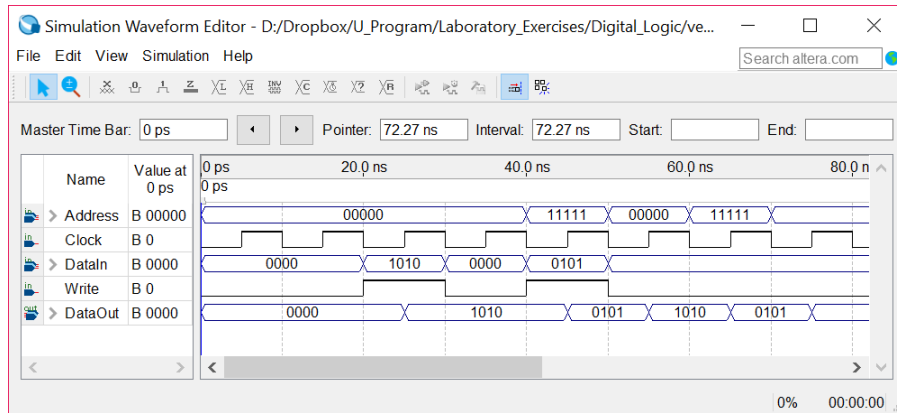


Figure 4: An example of simulation output.

Part II

Now, we want to realize the memory circuit in the FPGA on your DE-series board, and use slide switches to load some data into the created memory. We also want to display the contents of the RAM on the 7-segment displays.

1. Make a new Quartus project which will be used to implement the desired circuit on your DE-series board.
2. Create another VHDL file that instantiates the *ram32x4* module and that includes the required input and output pins on your DE-series board. Use slide switches SW_{3-0} to provide input data for the RAM, and use switches SW_{8-4} to specify the address. Use SW_9 as the *Write* signal and use KEY_0 as the *Clock* input. Show the address value on the 7-segment displays $HEX5 - 4$, show the data being input to the memory on $HEX2$, and show the data read out of the memory on $HEX0$.
3. Test your circuit and make sure that data can be stored into the memory at various locations.

Part III

Instead of creating a memory module subcircuit by using the IP Catalog, we can implement the required memory by specifying its structure in VHDL code. In a VHDL-specified design it is possible to define the memory as a multidimensional array. A 32×4 array, which has 32 words with 4 bits per word, can be declared by the statement

```
TYPE mem IS ARRAY(0 TO 31) OF STD_LOGIC_VECTOR(3 DOWNTO 0);
SIGNAL memory_array : mem;
```

In an FPGA such an array can be implemented either by using the flip-flops that each logic element contains or, more efficiently, by using the built-in memory blocks. The Quartus Help provides other examples of VHDL code that show how memory can be specified (search in the Help for “Inferred memory”).

Perform the following steps:

1. Create a new project which will be used to implement the desired circuit on your DE-series board.
2. Write a VHDL file that provides the necessary functionality, including the ability to load the RAM and read its contents as was done in Part II.
3. Assign the pins on the FPGA to connect to the switches and the 7-segment displays.
4. Compile the circuit and download it into the FPGA chip.
5. Test the functionality of your design by applying some inputs and observing the output.

Part IV

The SRAM block in Figure 1 has a single port that provides the address for both read and write operations. For this part you will create a different type of memory module, in which there is one port for supplying the address for a read operation, and a separate port that gives the address for a write operation. Perform the following steps.

1. Create a new Quartus project for your circuit. To generate the desired memory module open the IP Catalog and select the *RAM: 2-PORT* module in the Basic Functions > On Chip Memory category. As shown in Figure 5, choose With one read port and one write port in the category called How will you be using the dual port ram?

Configure the memory size, clocking method, and registered ports the same way as Part II. As shown in Figure 6 select I do not care (The outputs will be undefined) for Mixed Port Read-During-Write for Single Input Clock RAM. This setting specifies that it does not matter whether the memory outputs the new data being written, or the old data previously stored, in the case that the write and read addresses are the same during a write operation.

Figure 7 shows how the memory words can be initialized to specific values. It makes use of a feature that allows the memory module to be loaded with data when the circuit is programmed into the FPGA chip. As shown in the figure, choose the setting Yes, use this file for the memory content data, and specify the filename *ram32x4.mif*. An example of a MIF file is provided in Figure 8. You can also learn about the format of a *memory initialization file* (MIF) by using the Quartus Help. You will need to create a MIF file like the one in Figure 8 to test your circuit. Finish the Wizard and then examine the generated memory module in the file *ram32x4.vhd*.

2. Write a VHDL file that instantiates your dual-port memory. To see the RAM contents, add to your design a capability to display the content of each four-bit word (in hexadecimal format) on the 7-segment display *HEX0*. Use a counter as a read address, and scroll through the memory locations by displaying each word for about one second. As each word is being displayed, show its address (in hex format) on the 7-segment displays *HEX3–2*. Use the 50 MHz clock, *CLOCK_50*, and use *KEY₀* as a reset input. For the write address and corresponding data use switches *SW_{8–4}* and *SW_{3–0}*. Show the write address on *HEX5–4* and show the write data on *HEX1*. Make sure that you properly synchronize the slide switch inputs to the 50 MHz clock signal.
3. Test your circuit and verify that the initial contents of the memory match your *ram32x4.mif* file. Make sure that you can independently write data to any address by using the slide switches.

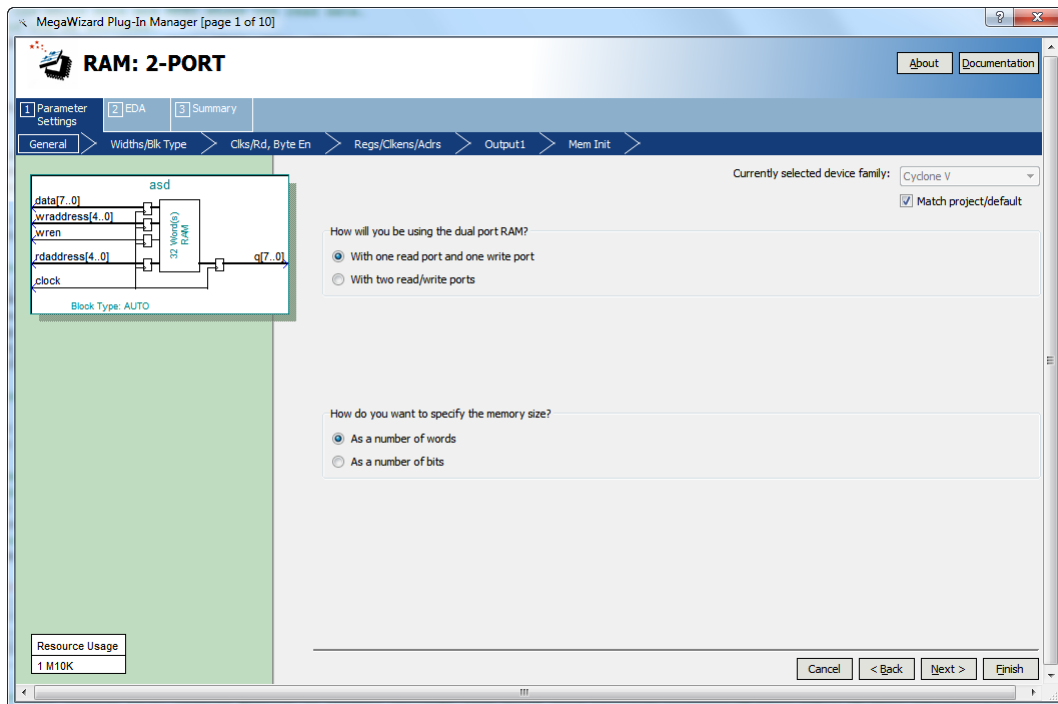


Figure 5: Configuring the two input ports of the RAM.

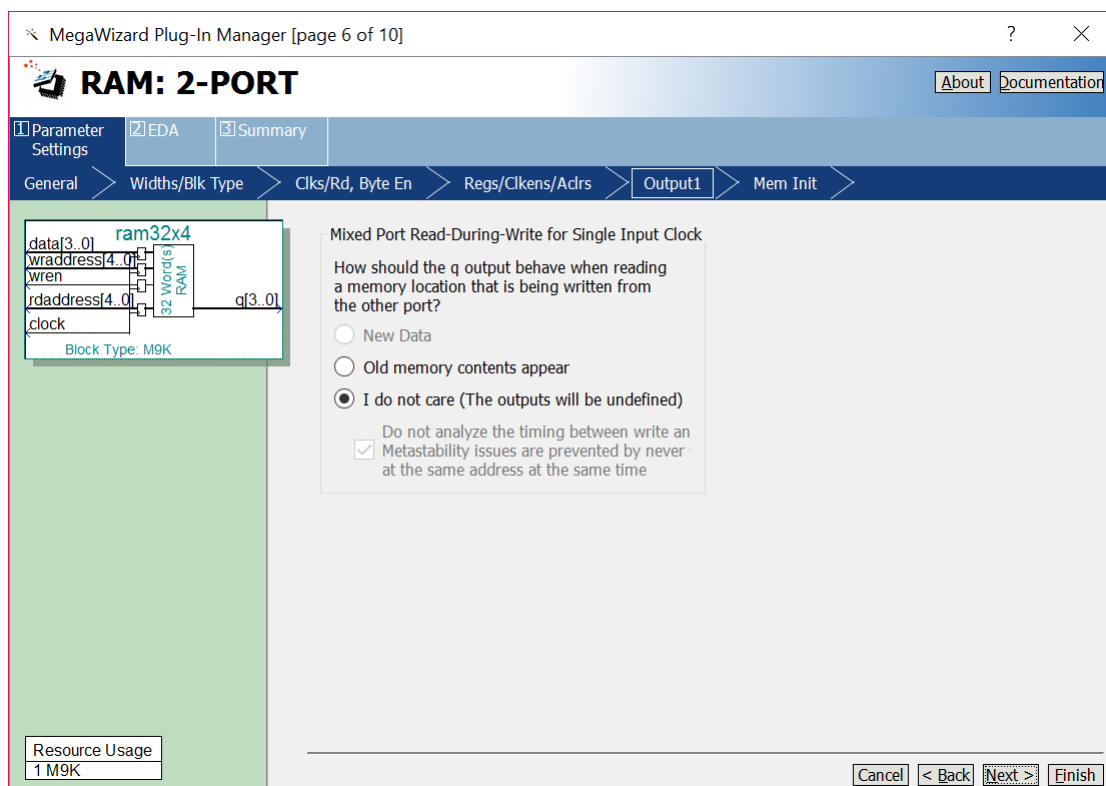


Figure 6: Configuring the output of the RAM when reading and writing to the same address.

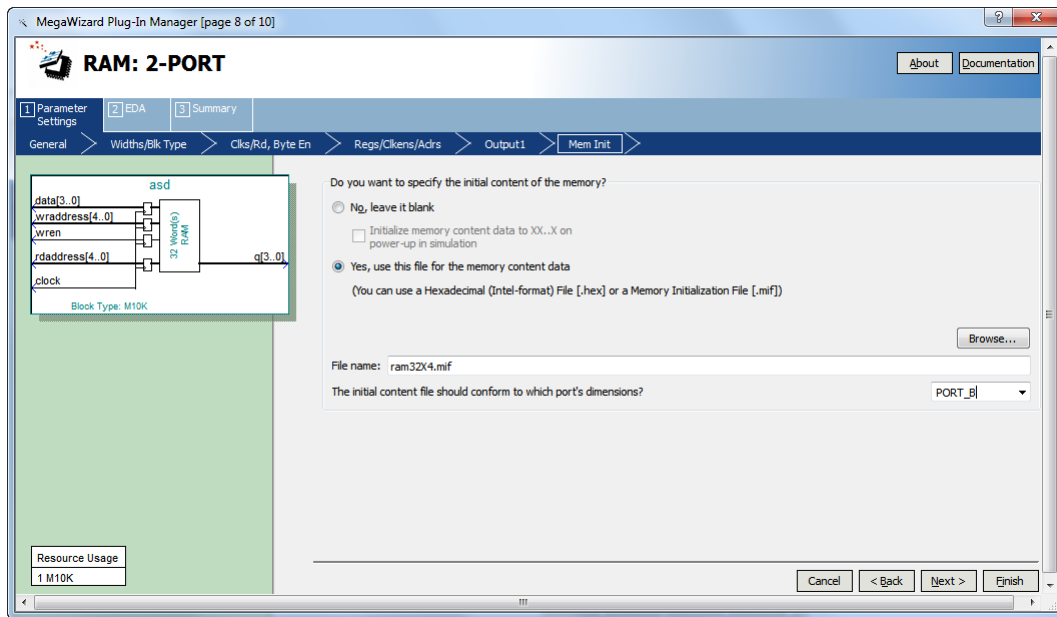


Figure 7: Specifying a memory initialization file (MIF).

```

DEPTH = 32;
WIDTH = 4;
ADDRESS_RADIX = HEX;
DATA_RADIX = BIN;
CONTENT
BEGIN

0 : 0000;
1 : 0001;
2 : 0010;
3 : 0011;
... (some lines not shown)
1E : 1110;
1F : 1111;

END;

```

Figure 8: An example memory initialization file (MIF).